

THE PRACTICAL CLAUDE PLAYBOOK

The Practical Claude Playbook

From Your First Chat to Autonomous Systems (2026)

Why this guide exists

Most people use Claude like a smarter search box. They ask it a question, copy the answer, and move on. That works, and it also leaves about 90 percent of the value on the table.

The people getting real leverage from Claude in 2026 are not better at writing prompts. They understand one thing the casual user does not: Claude is not a chatbot. It is an engine that can read your files, run tools, build deliverables, write and ship code, and do all of it on a schedule while you do something else.

This guide takes you from typing your first question to running systems that do real work without you in the loop. You do not need to code to start, and you will have a clear path for the day you want to go further.

Who this is for

Founders, operators, and business owners who want Claude to actually move the needle, not just answer questions. If you run a company, manage a team, or carry the operational load yourself, this is written for you. No engineering background assumed.

How to use this guide

It is built in three tiers. Jump to where you are.

TIER	YOU ARE HERE IF...	YOU WILL LEAVE ABLE TO...
Part 1 - Beginner	You have used Claude a little, or not at all	Get your first real wins this week and understand the whole landscape
Part 2 - Intermediate	You use Claude regularly but stick to chat	Pick the right tool every time and build workflows you reuse
Part 3 - Advanced	You want Claude doing work unattended	Automate operations and know what is worth building yourself vs handing off

One note on timing. Claude's products move fast. Everything here was accurate as of mid-2026. Where a price or limit matters to a decision, confirm the current number before you commit.

The one idea to take from this entire guide

One engine, many doors.

Anthropic builds the intelligence, the underlying models that do the thinking. Then it ships that same intelligence behind different doors, each one shaped for a different kind of person and a different level of independence.

- **The chat door:** For thinking out loud with a human steering every step.
- **The Claude Code door:** For engineers working inside a codebase.
- **The Cowork door:** For non-technical operators who want work done on their own desktop, no terminal required.
- **The API door:** For developers building Claude into their own products and automations.

Same brain. Different hands. The model answering you in a chat window is the same one refactoring a codebase or building a spreadsheet on someone's desktop.

Once this clicks, every "which tool should I use?" question answers itself. You only ever ask two things:

1. Who is doing the work? You, in the loop? An engineer? A non-technical teammate? Your own software?
2. How much autonomy does the task need? A single answer you review, or a multi-step job that runs on its own?

Hold those two questions in your head as you read. The rest of this guide is just teaching you to answer them well.

CONTENTS

Table of Contents

Three levels, thirty sections, one climb.

Beginner		PAGE
01	What Claude Actually Is (and Isn't)	5
02	The Claude Universe: Every Way to Use It	7
03	Claude.ai Deep Dive: Your First Real Workspace	10
04	Claude on Your Desktop and Phone	13
05	The Beginner's Prompting Playbook	15
06	10 Everyday Things to Use Claude For	17
07	The 7 Beginner Mistakes (and the Fixes)	19
08	Which Plan Should You Start On?	21
09	Staying Safe and Smart with Claude	23
10	Your 7-Day Claude Starter Plan	25

Intermediate		PAGE
11	Meet Claude Cowork	27
12	Cowork Workflows to Steal	29
13	Choosing Your Model: Opus, Sonnet, Haiku	31
14	Connecting Claude to Your Tools	33
15	Build Once, Reuse Forever: Your First Skill	35
16	Scheduling and Routines	37
17	Giving Claude a Memory: Context That Sticks	39
18	Build Your Own Assistant	41
19	Managing Usage, Cost, and Context	43
20	From Operator to Builder	45

Part 3 - Advanced

Claude Code, orchestration, and systems that run themselves.

Advanced

PAGE

21	Meet Claude Code: The Most Powerful Door	47
22	Claude Code Core Concepts	49
23	Creating Custom Workflows: Skills and Subagents	51
24	Orchestrating Agents at Scale	53
25	Extending Claude Code: Plugins, CLIs, and MCP	55
26	Integration Power-Ups: X + Y = Superpower	57
27	Building Systems That Run Themselves	59
28	The Business Operating System	61
29	Claude Code in the Real Business	63
30	Your Roadmap From Here	65

01

THE FOUNDATION

What Claude Actually Is (and Isn't)

The foundation every other chapter in this guide stands on.

BOTTOM LINE

Claude is a frontier AI assistant, not a chatbot and not a search engine. It reads, writes, reasons, codes, and can act on your behalf. Once you understand what it is genuinely good at and where it still needs a human, everything else gets easier.

01 Start here: it is not a chatbot

Most people meet Claude through a chat box, so they file it next to a search engine or a customer-service bot. That single framing is the biggest reason people underuse it. Claude is a general-purpose AI assistant built by Anthropic, an AI safety company. The same underlying intelligence can draft a proposal, analyze a messy spreadsheet, write and fix software, summarize a 200-page contract, or carry out a multi-step task on your computer while you do something else. The chat box is just the easiest door in.

02 What Claude is genuinely good at

- **Writing and editing.** Widely considered the strongest of the major models for natural, on-brand writing, long-form drafting, and tightening rough copy.
- **Reasoning and analysis.** It works through multi-step problems, reads long documents, and explains its thinking — holding up to a million tokens of context, roughly several thick books at once.
- **Coding.** Among the best in the world at writing and fixing software, scoring around 87 percent on a standard real-world engineering benchmark. You do not need to code to benefit.
- **Comparative accuracy.** In a 2026 long-form factuality test it made things up far less often than a leading competitor. Not flawless, but relatively careful.
- **Acting, not just answering.** Newer versions use tools, browse the web, work with your files, and complete tasks — not only return text.

03 One engine, many doors

Anthropic builds the intelligence — a family of models — then ships that same intelligence behind several doors, each shaped for a different person and a different level of independence. The chat website, a desktop app, a phone app, a coding tool, an autonomous desktop agent, and a developer interface all run the same brain. You choose the door by two things: who is doing the work, and how much you want Claude to act on its own.

04 What Claude is NOT

- **Not a search engine.** Trained on data up to a cutoff date, so it does not know today's news by default, though it can search the web when asked. Treat it as a reasoning engine, not a live index.
- **Not always right.** It can state wrong things with total confidence — hallucination. Verify anything that matters: names, numbers, quotes, sources.
- **Not deterministic.** Ask the same question twice and you may get two different answers. Normal for this kind of AI, and why high-stakes tasks eventually get structured.
- **Not a human, and not magic.** No memory of you between conversations unless you set it up, no instinct for your business unless you give it context, no judgment to spare you from checking its work.

WHY THIS MATTERS

The people who get the most out of Claude are not better at writing prompts. They understand what it is — a powerful, context-hungry reasoning engine — and what it is not, an oracle. They feed it context, they verify what matters, and they pick the right door for the job. That is the entire game.

■ Key takeaways

- 01 Claude is a frontier AI assistant from Anthropic — not a chatbot, not a search engine.
- 02 Strongest at writing, reasoning over long documents, and coding, and comparatively careful with facts.
- 03 One engine, many doors: the same intelligence reaches you through chat, desktop, mobile, a coding tool, an autonomous agent, and a developer interface.
- 04 Real limits — a knowledge cutoff, occasional confident mistakes, non-deterministic answers. Give it context and verify what matters.

05

THE CRAFT

Prompting Playbook

Six moves to better answers

Six moves that make every answer dramatically better.

BOTTOM LINE

Great results do not come from clever wording. They come from briefing Claude the way you would brief a sharp new hire: give it context, tell it exactly what you want, show it an example, and let it ask questions. Master these six moves and you will outperform most people using the same tool.

The biggest mistake beginners make is treating Claude like a vending machine: type a short request, expect a perfect answer. Treat it instead like a brilliant assistant on their first day — capable, but it knows nothing about you, your business, or what good looks like until you tell it.

01 Give it context — the biggest lever

Before the request, tell Claude what it needs to know: who you are, who the output is for, any relevant background. The same question with context attached produces a completely different answer. If the context lives in a document, upload it; if it is ongoing, put it in a Project so you never repeat yourself.

02 Ask it to ask you questions first

One line changes everything: **"Before you answer, ask me any questions you need to do this well."** Instead of guessing and handing you something generic, Claude surfaces the gaps in your request and fills them. This single habit prevents most bad first drafts.

03 Set the role and the format

Tell Claude who to be ("act as my marketing assistant," "you are a careful financial analyst") and exactly how you want the output — length, structure, tone. "A 120-word friendly email with one call to action" beats "write an email" every time. Vague in, vague out.

04 Show an example

If you have a sample of what good looks like — a past email you liked, a report in the right format — paste it in and say "match this style." Examples teach Claude your taste faster than any description, and stop output from sounding generic.

05 Iterate, do not restart

The first answer is a draft, not a verdict. Stay in the same conversation and steer: "shorter," "more direct," "drop the second paragraph," "warmer tone." Claude keeps the context and refines. Starting a brand-new chat throws away everything it just learned.

06 Make it check its work

For anything that matters, add "review your answer for mistakes before finalizing" or "double-check these numbers." Claude is comparatively careful, but it can still be confidently wrong. Asking it to verify catches a surprising number of errors — and you still confirm the important facts yourself.

07 Weak prompt vs strong prompt

WEAK

“Write a marketing email.”

STRONG

“You are my marketing assistant for a boutique accounting firm. Write a friendly cold email under 120 words to small-business owners offering a free bookkeeping review, with one clear call to action. Match the voice of this example: [paste]. Before you write, ask me anything you need.”

08 The reusable skeleton: R-C-T-F-E

Five parts, in order — then close with one request.

R	ROLE	who Claude should be
C	CONTEXT	the background and the files
T	TASK	exactly what you want done
F	FORMAT	length, structure, and tone
E	EXAMPLE	a sample of good output

Then close with: “Ask me questions before you start.”

■ Key takeaways

- 01 Brief Claude like a new hire, not a vending machine. Context is the single biggest lever.
- 02 Add “ask me questions first” to almost every real request; it kills generic drafts.
- 03 Set the role and format, and show an example to teach Claude your taste.
- 04 Iterate in the same conversation instead of restarting, and make it check its work on anything important.
- 05 Remember R-C-T-F-E: Role, Context, Task, Format, Example.

11

INTERMEDIATE

Meet Cowork

Claude stops answering and starts acting

Claude running as an autonomous assistant on your desktop.

BOTTOM LINE

You give Cowork a goal in plain English, it does the work, and it asks permission at key moments. No coding required. This is the jump from 'Claude helps me think' to 'Claude does the work while I think about something else.'

01 What Cowork actually is

In a normal chat you are in a back-and-forth conversation. Cowork changes that. You give Claude a goal, point it at your computer — a folder, a file, an app — and it executes step by step using tools to open files, read content, write outputs, and organize results. You see what it is doing and approve or redirect the decisions that matter.

02 The plan-to-action loop

1. Give Cowork a goal: 'Take all the feedback PDFs in this folder, extract the key themes, and build a summary spreadsheet — one row per document.'
2. Cowork builds a plan and shows it to you before doing anything.
3. You confirm the plan, or adjust it.
4. It executes step by step, flagging decisions that need your input.
5. You review the output at the end.

This loop keeps you in control without requiring you to do the work.

03 The sandboxed VM

Cowork operates inside a sandboxed virtual machine — a contained computer environment:

- **Isolated.** What it does in the sandbox does not affect your real computer unless you explicitly move something out.
- **Safe to experiment.** If something goes wrong, nothing is permanently broken. Reset and start fresh.

04 Ask Before Acting vs Act Without Asking

- **Ask Before Acting.** Cowork pauses before significant steps and waits for your approval. Best when you are learning, the task is new, or the stakes are high.
- **Act Without Asking.** Cowork completes the full task and only surfaces something if it genuinely cannot proceed. Best for routine, well-understood tasks you have run before.

Start with Ask Before Acting until you trust how Cowork handles your type of task.

05 Setting up Cowork

1. Open the Claude Desktop app.
2. Select Cowork from the mode options.
3. Point it at a folder you want to give it access to, or leave it at the default sandbox.
4. Describe your goal in plain English and confirm the plan.

GOOD FIRST TASK

'Take this folder of email threads, summarize the key decisions from each one, and put them in a table with columns for: thread subject, decision made, and next action.' Paste the folder path, run it in Ask Before Acting mode, and watch what it does. That 10-minute demo teaches more about Cowork than reading about it.

06 What Cowork is good at vs its limits

GOOD AT	LIMITS
File organization and renaming	Needs file access through paths you give it
Reading many documents and synthesizing them	Complex multi-app workflows work best on the API (Advanced section)
Building spreadsheets and reports from source material	Tasks requiring your real credentials or logged-in sessions need care
Batch operations on many files at once	

KEY MINDSET

Cowork is not magic. It is a precise, literal assistant. The better your goal is defined, the better the output. If you would not know how to explain the task to a new hire in two paragraphs, break it down further before giving it to Cowork.

■ Key takeaways

- 01 Cowork runs Claude as an autonomous assistant. You define the goal, it executes, you review.
- 02 The plan-confirm-execute-review loop keeps you in control without doing the work.
- 03 Start with Ask Before Acting. Switch to Act Without Asking once you trust the task.
- 04 The sandboxed VM means nothing permanent happens to your machine without your say.
- 05 Best for file work, batch processing, document synthesis, and spreadsheet generation.

14

TOOL CONNECTIONS

Connecting Your Tools

Claude plugged into your work stack

Gmail, Calendar, Drive, Slack, Notion and more.

BOTTOM LINE

Claude becomes dramatically more useful when it can read from and write to the tools you already work in. This section covers the three ways to connect: built-in integrations, Claude in Chrome, and the Model Context Protocol.

BUILT-IN CONNECTORS Gmail, Calendar, Drive, Slack, Notion	CLAUDE IN CHROME Any tab, any site, no copy-paste	MCP Connect virtually any tool to Claude
---	---	--

01 Built-in connectors

CONNECTOR	WHAT CLAUDE CAN DO
Gmail	Read, draft, and send emails. Triage your inbox. Draft replies in your voice.
Google Calendar	See your schedule. Draft meeting agendas. Find free time slots.
Google Drive / Docs	Read documents and sheets. Update Docs. Save outputs directly.
Slack	Read channel history. Draft messages. Post updates to threads.
Notion	Read databases and pages. Write summaries back into pages.

These connect through the Integrations panel in Claude Settings. Once connected, Claude can pull from these sources automatically when you ask for context -- no copy-paste, no switching tabs.

ONCE CONNECTED

Claude can check your Gmail, see what is on your calendar, read a Drive doc, or pull from a Slack thread -- all within a single conversation. You do not need to describe what is already in those tools. Claude reads the source directly.

02 Claude in Chrome

The Chrome extension brings Claude into any browser tab with the page content already loaded:

- **Summarize an article** you are reading without copying it out of the tab.
- **Draft a Gmail reply** without switching to a separate Claude window.
- **Analyze a report** in Google Sheets or Docs without leaving it.
- **Get quick answers** about any page's content while staying in the tab.

The Chrome extension works with any website, not just Google tools. If the page is open, Claude can see it and work with it.

03 MCP: the deeper connection

The Model Context Protocol (MCP) is the standard that lets Claude connect to virtually any tool -- including ones Anthropic did not build. Think of MCP like USB-C for AI tools: a common plug that any compatible tool can use.

MCP servers exist for dozens of professional tools. If the built-in connectors do not include something you use, there is likely an MCP server for it already. If your team has a developer, they can build a custom MCP server for any internal tool in a few hours.

- **GitHub, Jira, Asana** for project and engineering workflows.
- **Salesforce, HubSpot** for CRM data and pipeline context.
- **Custom internal APIs** and databases your team already runs.

USB-C FOR AI

MCP is the standard connector that turns Claude into a hub for your entire stack. Any tool with an MCP server can talk to Claude. The list grows every month and most of the common professional tools are already covered.

04 What a connected week looks like

Four examples of how connectors change a working day:

- | | |
|-----------------------|---|
| MORNING | Summarize the 10 most important emails from the last 24 hours and tell me what needs a response today. (Gmail connector) |
| BEFORE MEETING | Read my calendar and draft a 3-point agenda for my 2pm client check-in based on the notes in the shared Drive doc. (Calendar + Drive) |
| AFTERNOON | Take the feedback from this morning's Slack thread and turn it into a structured brief for the design team. (Slack) |
| END OF DAY | Save today's decisions and action items to the project Notion page. (Notion) |

START WITH ONE

Pick the tool you use most -- Gmail, Drive, or Slack -- and connect it first. One integration that saves 20 minutes a day is worth more than five integrations set up and never used. Get value from one before adding more.

05 Setting up your first connector

1. Open Claude Settings and go to the Integrations panel.
2. Click the tool you want to connect and follow the authorization prompt. It takes about two minutes.
3. Start a new conversation, mention the tool by name, and ask Claude to pull context from it. The connection is live immediately.

■ Key takeaways

- 01 Built-in connectors (Gmail, Calendar, Drive, Slack, Notion) let Claude read from and write to your actual tools -- no manual copying needed.
- 02 The Chrome extension brings Claude into any browser tab with the page already loaded. It works on any site, not just Google tools.
- 03 MCP extends Claude to virtually any tool through a common standard. If the built-in list does not cover a tool you use, look for an MCP server.
- 04 Start with one connector. One integration used consistently beats five set up and abandoned.
- 05 A connected Claude does not need you to re-describe what is already in your tools. It reads the source directly, every time.

CONNECTED WORKFLOW

The biggest shift is not the time saved per task. It is that Claude stops being a separate tool you switch to and becomes part of the workflow you are already in. That changes how often you reach for it.

MEET CLAUDE CODE

Meet Claude Code: AI That Builds The Most Powerful Door

And why you don't need to be a coder to open it.

BOTTOM LINE

Claude Code is Claude in a terminal or code editor, with the ability to read, write, and edit files across an entire project. It builds software, fixes bugs, designs systems, and runs automations. You do not need to be a programmer to start -- but it rewards people willing to engage at a technical level.

WHAT IT IS	NO CODE NEEDED	7 CAPABILITIES	4-STEP SETUP
Claude in your editor with full file access	Describe in English, Claude builds it	Code, fix, refactor, test, automate	Install and run in about 10 minutes

01 What Claude Code actually is

Claude Code is a command-line tool and IDE integration that gives Claude access to your file system, your terminal, and your codebase. Instead of responding in a chat window, it works inside your development environment -- reading files, writing code, running commands, and iterating on results.

The key difference from the chat interface: Claude Code can see and edit many files at once, run your code to test it, and make changes across an entire project -- not just answer questions about it.

02 You do not need to be a coder

That is not marketing language. Non-technical people use Claude Code to:

1. Build working prototypes from a plain-English description -- no syntax knowledge required.
2. Automate repetitive file operations and data processing tasks.
3. Create scripts that run on a schedule without any manual work.
4. Build internal tools such as dashboards, reports, and automators without a development team.

The pattern: describe what you want in plain English, review what Claude builds, test it, iterate. You are the product manager; Claude is the developer.

03 What it can do

- **Write code from scratch:** Describe what you want; it writes the code and runs it for you.
- **Read an existing project:** Point it at any codebase -- ask questions, add features, fix things.
- **Fix bugs:** Describe the problem or paste the error; Claude diagnoses and patches it.
- **Refactor and clean up:** Improve code structure and clarity without changing what it does.
- **Write tests:** Generate automated tests for existing functionality with no manual effort.
- **Build automations:** Scripts, pipelines, and scheduled tasks that run independently.
- **Work across many files:** Holds the full project in view and makes consistent changes everywhere.

04 Four-step setup

1. Install Node.js (free, at nodejs.org) if you do not already have it. It is the runtime Claude Code runs on.
2. Open a terminal. On Mac: Terminal app. On Windows: Command Prompt or PowerShell.
3. Run: **`npm install -g @anthropic-ai/claude-code`**
4. Navigate to your project folder, then run: **`claude`**

That is the complete setup. Claude Code opens in the terminal connected to your folder.

05 Your first run

Once Claude Code is running, try this:

1. Tell it what you want to build. For example: "Read all the CSV files in this folder, combine them, and output a summary." Plain English is enough.
2. Watch what it does: it reads your folder, proposes a plan, writes the code, and runs it. You see every step.
3. Review the output, test it, and iterate. Ask follow-up questions or request changes in plain English.

06 Golden rules for non-coders

- **Work in a test folder first:** Do not point Claude Code at your main business files until you understand how it behaves.
- **Read what it proposes before saying yes:** The plan step is your quality gate. Claude shows its work -- use that.
- **Save working versions:** Use Git or copy a working version before asking Claude to make changes.
- **Ask Claude Code to explain what it did:** "Walk me through what that script does" is a legitimate and useful instruction.

WELCOME TO ADVANCED

Claude Code is the first tool here. It is also the most powerful one.

YOU ARE THE PRODUCT MANAGER

Describe what you want in plain English, review the plan Claude proposes, test the output, and iterate. That relationship requires no code knowledge. Claude is the developer. You hold the brief.

■ Key takeaways

- 01 Claude Code is Claude in a terminal or editor, working across your full project. It reads, writes, runs, and iterates -- not just answers questions.
- 02 Non-technical users build real, working tools with it. Describe what you want, review the plan before approving, test the output, and iterate from there.
- 03 Setup takes about 10 minutes: install Node.js, run the npm install command, navigate to your folder, and type claude. That is the full process.
- 04 Start in a test folder. Read every plan before saying yes. Save working versions. Ask Claude to explain what it built. These four habits prevent almost every beginner mistake.

TERMINAL QUICK-START

- 01 **npm install -g @anthropic-ai/claude-code**
 - 02 **cd your-project-folder**
 - 03 **claude**
 - Claude Code opens. Tell it what you want to build.
-

THE CEILING JUST MOVED

Most people never get this far. The ones who do find that Claude Code closes the gap between knowing what they want and having it built. Start in a test folder. Run the command. See what happens.

26

INTEGRATION POWER-UPS

Integration Power-Ups

Claude + Your Stack

What happens when you combine Claude with the right tools.

BOTTOM LINE

Each integration is useful alone. The real leverage comes from combining them: Claude Code plus Firecrawl, plus NotebookLM, plus Playwright, plus Graphify and Obsidian. The pattern is always Claude as the intelligent layer on top of purpose-built tools.

FIRECRAWL Scraping pipelines at scale	NOTEBOOKLM Near-infinite cited memory	PLAYWRIGHT Real browser automation	GRAPHIFY + OBSIDIAN A living second brain
--	--	---	--

01 Claude Code + Firecrawl = scraping at scale

- **Unlocks:** Crawl a site, extract structured data, and process the results automatically -- no brittle scraper code.
- **How it works:** Claude Code calls the Firecrawl CLI, which crawls and returns clean output. Claude Code then filters, categorizes, and summarizes it.
- **Real use case:** Point it at 50 competitor sites; extract each services, pricing, and about page into one comparison table, no manual browsing.

02 Claude Code + NotebookLM = near-infinite memory

- **Unlocks:** The context window has limits; NotebookLM holds a much larger body of knowledge and answers with citations.
- **How it works:** Claude Code queries a notebook via the CLI, gets a cited answer back, and uses it in the current task without loading everything into context.
- **Real use case:** Build a research-backed report: query the notebook for findings, get cited answers, synthesize a document without the full archive in context.

03 Claude Code + Playwright = browser automation

- **Unlocks:** Control a real browser -- navigate, fill forms, click, extract content, screenshot -- then decide based on what it sees.
- **How it works:** Claude Code drives Playwright through the CLI. Playwright launches a browser, performs the actions, and returns the results.
- **Real use case:** An automated intel loop: every Monday, browse 10 competitor sites, capture pricing or feature changes, and email you a diff report.

04 Graphify + Obsidian = second brain

- **Unlocks:** Turn Claude Code's outputs -- diagrams, notes, research, decision logs -- into a permanent, visual, linked knowledge base.
- **How it works:** Graphify generates visual diagrams from code or ideas; those plus session notes save into an Obsidian vault that grows into a linked graph.
- **Real use case:** Every session that produces a real decision saves to the vault with architecture diagrams -- an always-current technical knowledge base.

THE PATTERN BEHIND THE COMBOS

Claude reasons and decides, external tools execute the specialized work, and Claude synthesizes the results. This scales to any tool with a CLI or an MCP server -- the only question is what specialized tool handles a step better than Claude alone, and can Claude call it.

■ Key takeaways

- 01 Four high-value combos: Firecrawl (scraping), NotebookLM (large cited memory), Playwright (browser automation), Graphify + Obsidian (second brain).
- 02 The pattern is always Claude as the intelligent layer calling specialized tools for the specialized work.
- 03 Any tool with a CLI or an MCP server can join this pattern.
- 04 The real leverage comes from combinations, not individual tools.

NEXUSPOINT

You just read 6 of 30. Get the full playbook.

All 30 sections. Free.

This is a sample. The complete Practical Claude Playbook runs thirty sections across three levels, from your first chat to systems that run themselves. Grab the whole thing in one click.

GET THE FULL GUIDE

All 30 sections, free

Download the complete playbook at the link below. No fluff, just the system we use to put Claude to work in real businesses.

FREE SYSTEMS AUDIT

See where Claude saves you the most

Want it mapped to your operation? Book a free teardown. We find the three highest-leverage places Claude cuts your time or cost. No pitch, no commitment.

GET IT HERE

nexus-point.co